

# MongoDB Food Delivery Aggregation Assignment

Database: **foodDB** | Collections: **restaurants**, **orders** | Tool: mongosh

Name: \_\_\_\_\_ Roll No.: \_\_\_\_\_ Date: \_\_\_\_\_

**Note:** The outputs below are the expected results for the sample data in Questions 1 and 2. Run the commands in mongosh to get your own screenshots. In Question 5 the statuses are assigned randomly, so your final count can differ from the sample shown.

## Question 1: Create the restaurants collection with 5 documents

*Create a collection named 'restaurants' and insert 5 documents, each a different restaurant with the fields name, cuisine, location and average\_rating.*

### Commands

```
use foodDB
db.createCollection("restaurants")

db.restaurants.insertMany([
  { _id: "R101", name: "Spice Route", cuisine: "Indian",
    location: "Ahmedabad", average_rating: 4.5 },
  { _id: "R102", name: "Dragon Wok", cuisine: "Chinese",
    location: "Surat", average_rating: 4.3 },
  { _id: "R103", name: "Bella Napoli", cuisine: "Italian",
    location: "Vadodara", average_rating: 4.6 },
  { _id: "R104", name: "Thali House", cuisine: "Gujarati",
    location: "Ahmedabad", average_rating: 4.8 },
  { _id: "R105", name: "Pizza Palace", cuisine: "Pizza",
    location: "Rajkot", average_rating: 4.4 }
])
```

### Output

```
{ acknowledged: true,
  insertedIds: { '0': 'R101', '1': 'R102', '2': 'R103', '3': 'R104', '4': 'R105' } }
```

**Explanation:** Simple string ids (R101 to R105) are used as `_id` so they are easy to read and to refer to from the orders collection. MongoDB would otherwise create an ObjectId automatically.

## Question 2: Design and fill the orders collection

*Design an 'orders' collection for a food delivery app where each order has user\_id, restaurant\_id, items (an array of objects with item\_name and price), total\_amount and order\_date. Insert 3 sample orders.*

## Design

| Field         | Type             | Purpose                                                      |
|---------------|------------------|--------------------------------------------------------------|
| _id           | String           | Order id (ORD1, ORD2, ...)                                   |
| user_id       | String           | Customer who placed the order                                |
| restaurant_id | String           | Reference to restaurants._id                                 |
| items         | Array of objects | Each item has item_name and price, embedded inside the order |
| total_amount  | Number           | Sum of the item prices (in rupees)                           |
| order_date    | Date             | When the order was placed                                    |

**Design choice:** The items are embedded because they belong to one order and are always read together with it. The restaurant is stored as a reference (restaurant\_id) because a restaurant is shared by many orders.

## Commands

```
db.orders.insertMany([
  { _id: "ORD1", user_id: "U001", restaurant_id: "R101",
    items: [ { item_name: "Paneer Butter Masala", price: 260 },
              { item_name: "Butter Naan", price: 40 } ],
    total_amount: 300, order_date: ISODate("2026-09-22T13:15:00Z") },

  { _id: "ORD2", user_id: "U002", restaurant_id: "R104",
    items: [ { item_name: "Gujarati Thali", price: 350 },
              { item_name: "Buttermilk", price: 50 } ],
    total_amount: 400, order_date: ISODate("2026-09-23T19:40:00Z") },

  { _id: "ORD3", user_id: "U003", restaurant_id: "R101",
    items: [ { item_name: "Veg Biryani", price: 220 },
              { item_name: "Raita", price: 60 } ],
    total_amount: 280, order_date: ISODate("2026-09-24T12:05:00Z") }
])
```

## Output

```
{ acknowledged: true, insertedIds: { '0': 'ORD1', '1': 'ORD2', '2': 'ORD3' } }
```

## Question 3: Count the orders for each restaurant

*Write an aggregation query that calculates the total number of orders for each restaurant and displays the restaurant\_id with its order count.*

## Query

```
db.orders.aggregate([
  { $group: { _id: "$restaurant_id", order_count: { $sum: 1 } } },
  { $project: { _id: 0, restaurant_id: "$_id", order_count: 1 } },
  { $sort: { order_count: -1 } }
])
```

## Output

```
{ order_count: 2, restaurant_id: 'R101' }
{ order_count: 1, restaurant_id: 'R104' }
```

**Explanation:** \$group collects the orders that share the same restaurant\_id and adds 1 for every order (\$sum: 1). \$project renames the group key back to restaurant\_id, and \$sort lists the busiest restaurant first. R101 has 2 orders (ORD1 and ORD3) and R104 has 1 (ORD2).

## Question 4: Top 2 restaurants by average order amount

Generate a report using aggregation that finds the top 2 restaurants with the highest average order amount.

### Query

```
db.orders.aggregate([
  { $group: { _id: "$restaurant_id", avg_order_amount: { $avg: "$total_amount" } } },
  { $sort: { avg_order_amount: -1 } },
  { $limit: 2 },
  { $project: { _id: 0, restaurant_id: "$_id", avg_order_amount: 1 } }
])
```

### Output

```
{ avg_order_amount: 400, restaurant_id: 'R104' }
{ avg_order_amount: 290, restaurant_id: 'R101' }
```

**Explanation:** R104 has one order of Rs 400, so its average is 400. R101 has two orders (300 and 280), so its average is  $(300 + 280) / 2 = 290$ . \$sort puts the highest average first and \$limit keeps the top 2. With only 3 sample orders both restaurants appear; with more data, \$limit would cut the list to 2.

### Optional: show the restaurant name using \$lookup

```
db.orders.aggregate([
  { $group: { _id: "$restaurant_id", avg_order_amount: { $avg: "$total_amount" } } },
  { $sort: { avg_order_amount: -1 } },
  { $limit: 2 },
  { $lookup: { from: "restaurants", localField: "_id", foreignField: "_id", as: "r" } },
  { $unwind: "$r" },
  { $project: { _id: 0, restaurant_id: "$_id", name: "$r.name", avg_order_amount: 1 } }
])
// Output: { avg_order_amount: 400, restaurant_id: 'R104', name: 'Thali House' }
//         { avg_order_amount: 290, restaurant_id: 'R101', name: 'Spice Route' }
```

## Question 5: Add delivery\_status and count the pending orders

Refactor the orders collection to include a 'delivery\_status' field (pending, out for delivery, delivered). Update all existing orders to randomly assign one of these statuses, then count how many orders are 'pending'.

### Commands

```
db.orders.updateMany({}, [
  { $set: { delivery_status: {
    $arrayElemAt: [
      [ "pending", "out for delivery", "delivered" ],
      { $floor: { $multiply: [ { $rand: {} }, 3 ] } }
    ] } } }
])

// Check the assigned statuses
db.orders.find({}, { delivery_status: 1 })

// Count the pending orders
db.orders.countDocuments({ delivery_status: "pending" })
```

### Sample output (your statuses will be different because they are random)

```
{ acknowledged: true, matchedCount: 3, modifiedCount: 3 }
```

```
{ _id: 'ORD1', delivery_status: 'delivered' }  
{ _id: 'ORD2', delivery_status: 'pending' }  
{ _id: 'ORD3', delivery_status: 'out for delivery' }
```

1

**Explanation:** `updateMany( {}, ... )` updates every order. The update uses an aggregation pipeline: `$rand` gives a random number between 0 and 1 for each document, multiplying by 3 and using `$floor` turns it into 0, 1 or 2, and `$arrayElemAt` picks the status at that position. Because the choice is random, the pending count can be anywhere from 0 to 3 each time you run it.

### Alternative for older MongoDB versions (before 4.4.2, where `$rand` is not available)

```
const statuses = ["pending", "out for delivery", "delivered"];  
db.orders.find().forEach(o => {  
  db.orders.updateOne({ _id: o._id },  
    { $set: { delivery_status: statuses[Math.floor(Math.random() * 3)] } } });  
});
```